

TD 3 SR10/AI16

Sur deux séances

Partie 1 – préparation de l'environnement de développement

L'objectif de cette partie est la création d'un premier projet express (Framework node.js) pour la réalisation d'une application MVC. Avant d'installer express vérifiez si sur votre machine l'environnement Node.js et npm (gestionnaire des paquets de l'environnement d'exécution JavaScript Node.js) sont installés.

Les étapes à suivre :

- Installer le package **express-generator** (sur un terminal)

```
>npm install -g express-generator
```

```
>express -h
```

- Créer un projet express (EJS est choisi comme moteur de visualisation) :

```
> express --view=ejs myapp
```

```
create : myapp\
create : myapp\public\
create : myapp\public\javascripts\
create : myapp\public\images\
create : myapp\public\stylesheets\
create : myapp\public\stylesheets\style.css
create : myapp\routes\
create : myapp\routes\index.js
create : myapp\routes\users.js
create : myapp\views\
create : myapp\views\error.ejs
create : myapp\views\index.ejs
create : myapp\app.js
create : myapp\package.json
create : myapp\bin\
create : myapp\bin\www
```

Description de la commande express : génère un dossier qui a le même nom que le projet. Dans ce dossier, plusieurs sous dossiers et fichiers sont générés automatiquement. Le tableau ci-dessous décrit les différents éléments générés par express.

Dossier/fichier	Description
public	Les ressources statiques (html, css, javascript, images)
routes	Contient les scripts JS qui traitent les requêtes http
routes\index.js	Traiter les requêtes de l'url home (/)
routes\users.js	Traiter les requêtes de l'url (/users)
view\	Contient les templates ejs
view\error.ejs	Template (visuel) de la page d'erreur
view\index.ejs	Template (visuel) de la page accueil
app.js	Le point d'entrée d'une application express
package.js	Les dépendances
bin\www	Configurer et lancer le serveur http

- Les commandes pour tester cette application :
 1. Aller dans le dossier de l'application :
> cd myapp
 2. Installer les dépendances :
> npm install
 3. Lancer l'application (windows):
> SET DEBUG=myapp:* & npm start

Question :

Analyser les fichiers et les dossiers créés par express-generator afin de répondre aux questions suivantes :

- Quels fichiers et dossiers sont créés par express-generator ?
- À quoi sert le fichier app.js dans une application Express ?
- Comment les routes sont-elles ajoutées dans app.js ?
- Comment importer (require / import) et exporter (module.exports / export) les routes entre les fichiers et app.js ?
- Où sont définies les routes dans le projet (dans quels fichiers/dossiers) ?
- À quoi sert le dossier qui regroupe toutes les routes ?
- Comment définit-on une route en Express ?
- Comment une route permet-elle de traiter une requête sur une URL donnée ?
- Quels sont les paramètres d'une route (req, res, next) et à quoi servent-ils ?
- Quelle est la différence entre les autres méthodes HTTP (.get(), .post(), .put(), .delete(), etc.) et quand les utiliser ?
- À quoi sert .all() dans Express et dans quels cas l'utiliser ?
- Comment fonctionne la partie views avec EJS (rendu des pages, variables, templates) ?
- À quoi sert le dossier static (fichiers CSS, images, JS) et comment est-il configuré dans Express ?

Partie 2 – Création de la partie modèle

Dans cette partie, On ajoute un dossier « model » qui représente la partie modèle de l'architecture MVC de votre projet. Dans ce dossier, on commence d'abord par créer un script js (ici « db.js ») pour la connexion à la base de données MySQL :

```
var mysql = require("mysql");

var pool = mysql.createPool({
  host: "tuxa.sme.utc", //ou localhost
  user: "ai16pxxxx",
  password: "*****",
  database: "ai16pxxx"
});

module.exports = pool;
```

Par la suite, on doit créer pour chaque entité de notre modèle un fichier « js » qui regroupe les opérations de persistance CRUD (create, read, update, delete). Si on prend par exemple la table utilisateur (version simplifiée pour l'exemple) :

```
CREATE TABLE `Utilisateur` (  
  `email` varchar(25) NOT NULL,  
  `pwd` varchar(25) NOT NULL,  
  `nom` varchar(25) NOT NULL,  
  `prenom` varchar(25) NOT NULL,  
  `type` varchar(10) NOT NULL  
)
```

Un exemple de modèle pour cette table :

```
var db = require('./db.js');  
module.exports = {  
  read: function (email, callback) {  
  
    db.query("select * from Utilisateur where email= ?",email, function  
(err, results) {  
      if (err) throw err;  
      callback(results);  
  
    });  
  
  },  
  readall: function (callback) {  
    db.query("select * from Utilisateur", function (err, results) {  
      if (err) throw err;  
      callback(results);  
  
    });  
  
  },  
  areValid: function (email, password, callback) {  
    sql = "SELECT pwd FROM USERS WHERE email = ?";  
    rows = db.query(sql, email, function (err, results) {  
      if (err) throw err;  
      if (rows.length == 1 && rows[0].pwd === password) {  
        callback(true)  
      } else {  
        callback(false);  
      }  
    }  
  }  
}
```

```

    });

    },

    creat: function (email, nom, prenom, pwd, type, callback) {
        //todo
        return false;
    }
}
}

```

Questions :

- Développer l'ensemble des scripts du modèle de votre projet (utilisateur, offre, candidature, etc.).
- Remplacer les callbacks par des promesses, puis adopter async/await afin de rendre le code plus lisible, maintenable et moins sujet aux erreurs liées à l'imbrication.

Partie 3 – connecter le modèle, la vue et le contrôleur

Dans cette partie, nous allons créer des routes (intègrent des contrôleurs) qui vont traiter les requêtes http (de type get) et renvoyer une vue.

- L'ajout d'une route dans le contrôleur « routes/user.js ». Cette route a comme url : <http://localhost:3000/users/userslist> et elle retourne une vue qui affiche la liste de tous les utilisateurs dans un tableau (Array js).

```

.....
/* version avec modèle qui utilise un callback, il faut adapter le code en exploitant la
version avec promesse en utilisant await ou then et catch */
router.get('/userslist', function (req, res, next) {

    result=userModel.readall(function(result){

        res.render('usersList', { title: 'List des utilisateurs', users:
result });

    });

    });
.....

```

- L'ajout d'une vue « userList » (userList.ejs) :

```

<body>
  <h1>
    <%= title %>

```

```

</h1>
<table border>
  <thead>
    <tr>
      <th>Firstname</th>
      <th>Lastname</th>
      <th>Email</th>
    </tr>
  </thead>
  <tbody>
    <% users.forEach((user)=> { %>
      <tr>
        <td>
          <%= user.prenom %>
        </td>
        <td>
          <%= user.nom %>
        </td>
        <td>
          <%= user.email %>
        </td>
      </tr>
    <% }) %>
  </tbody>
</table>

</body>

```

Questions :

- Adapter et ajouter des routes pour traiter les requêtes http (get) de votre application.
- Utiliser vos templates html/css/bootstrap pour créer les vues EJS de votre projet qui permettent de visualiser les données de votre base de données (liste des utilisateurs, liste des offres, liste des organisations, etc.).

Ressources

- <https://github.com/mysqljs/mysql#pooling-connections>
- <https://ejs.co/>